

AI for Robotics - Project 1

Gavin Hill & Ryan Gale

1 Introduction

This project tasked students to write an AI algorithm to beat 5 variants of bomberman. the variants ranged from easy boards with no monsters, to challenging boards with multiple monsters. The monsters were also not consistent with "stupid" monsters that would move randomly, and "aggressive" monsters that would hunt down Bomberman. The main objective of each variant was to reach the exit goal without getting killed by a monster or by bomberman's own explosion. The main method of achieving artificial intelligence was via expectimax and the Bellman equation.

2 Code Implementations

2.1 do function

Our interactive character follows the do function with 5 main steps. Our first step uses numpy's vectorization to optimize lookup times for path planning during recursive calls in our expectimax's reward calculations. To do this we use a BFS search starting from the exit node and go through the whole world, giving us the distance to the end at any given cell in $O(1)$ time. Next the algorithm checks for winning conditions, or if bomberman can immediately escape. Then the algorithm checks for the opportunity to drop a bomb optimistically to either kill a monster or break a gating wall. Next, this algorithm will then see if any moves can be immediately pruned away if the move would cause immediate death. Finally, this algorithm enters the expectimax phase, where it rapidly recurses through the states till it reaches a world with the highest expected reward, returning the actions that would reach that state.

2.2 Precomputing distance

The code has a nice way of making precomputed distances to each cell by using BFS from the exit to each cell. This gives us the number of grid cells away each given cell is from the start. We then put this into a numpy vector such that each lookup is efficient at $O(1)$ time.

2.3 Calculating Terminal Conditions

This calculation is quite trivial. A lose is confirmed when bomberman's 2d pose is on the same square as any monster or any current blast cell from a bomb. A win is calculated used

a similar method, if bomberman's 2d pose matches the world exit cell 2d pose.

2.4 Bombing

Our bombing algorithm prioritizes bombing for multiple reasons. The first reason bombs are triggered is if going through a wall would have computed a faster time (i.e. less tiles walked total) bomberman will place a bomb in this tile. Additionally, bomberman will optimistically place bombs. optimistic bombs will be placed if one of these conditions is met: there is no path to the goal, there is a monster nearby, or if the neighbor wall blocks the way.

2.5 Initial Pruning

The code has the ability to prune bad paths. The algorithm looks for terminal states in which the `is_lose()` function returns a losing state and immediately discredits that as a legal move shaving future recursions and ultimately optimizing the total search time.

2.6 Expectimax

Our Expectimax algorithm models the bomber-man player as the max node and the monsters as chance nodes. it runs via a deterministic world. We first clone and make a sensed world which we then mutate up to n search depths. Each world is processed via the `sensed_world.next()` function. Then this new state is evaluated. We chose this implementation as it allows us to model the entire world accounting for variables such as bombs and individual monster patterns applied to the grid. To further optimize the expectimax time complexity, we have the algorithm only use a search depth once we are within 4 tiles from a monster. to further optimize the game, we decided that the algorithm would always cause bomberman to retreat if we are within 3 tiles, while this does extend games and lower our score, we decided that survivalability was of maximum importance and therefor this was considered a safer approach.

2.7 Tuning Behavior

In our implementation we added a pure evasion override feature if actively threatened by a monster. We found that starting this evasion when within 5 units from a monster gave the best results. We also added a parameter to tune when to start placing bombs relative to a monsters distance. Starting this within a radius of 6 units gave us the best results.

3 Experimental Evaluation

We used a multithreading python script to run 340 tests for the various variants of the game all with different starting conditions (no fixed seed). The results are:

```

● (.venv) orion@orion-ThinkPad-P1-Gen-7:~/Desktop/AI for Robotics/HWs/Project 1/RBE595/teamNN/project1$ python test_suite.py
variant 1 wins: 20/20
variant 2 wins: 20/20
variant 3 wins: 75/100
variant 4 wins: 62/100
variant 5 wins: 75/100

```

The test script for reference:

```

1
2 import subprocess
3 import sys
4 from concurrent.futures import ThreadPoolExecutor
5
6 variants = ["variant1.py", "variant2.py", "variant3.py", "variant4.py", "
    variant5.py"]
7 variant_wins = [0, 0, 0, 0, 0]
8 num_batches_per_variant = [4, 4, 20, 20, 20]
9 num_thread_workers = 5
10
11 def run_variant(variant_name):
12     try:
13         result = subprocess.run([sys.executable, variant_name],
14                                 capture_output=True, text=True, timeout=10)
15         output = result.stdout
16
17     except subprocess.TimeoutExpired as e:
18         output = e.stdout
19     # returns whether the game ended in a win
20     return 1 if "found the exit" in str(output) else 0
21
22 # run num_batches*num_thread_workers tests for each variant
23 for idx, variant in enumerate(variants):
24     for _ in range(num_batches_per_variant[idx]):
25         with ThreadPoolExecutor(max_workers=num_thread_workers) as
26         executor:
27             results = executor.map(run_variant, [variants[idx]]*
28             num_thread_workers)
29
30             for result in results:
31                 # record win rate
32                 variant_wins[idx] += result
33
34 print("variant " + str(idx+1) + " wins: " + str(variant_wins[idx])+
35       "/" + str(num_batches_per_variant[idx]*num_thread_workers))

```

Listing 1: Evaluation benchmark harness and batch variant test runner.